Proyecto 2 — Procesador RISC-V RV32I

Entrega E1 — Informe



Curso: Arquitectura de Computadoras

Integrantes: Gabriel Ormachea, Miguel Fernando Ucañani Tintaya, Fernando Aguilar
Fecha: 2 de Noviembre, 2026

Índice

Índice.....
1. Objetivo de E1.....
2. Pipeline Single-Issue de cinco etapas.....
   2.1 Las cinco etapas.....
   2.2 Datapath por etapas.....
   2.3 Unidad de control.....
3. Hazard Unit (cambios del Single-Issue).....
   3.1 Register File en flanco de bajada.....
   3.2 Forwarding (data hazard).....
   3.3 Stall (load-use).....
   3.4 Flush (control hazard: branch/jump).....
   3.5 Diagrama del datapath Single-Issue con Hazard Unit.....
4. Pruebas: cuatro programas y waveforms.....
   4.1 Programa sin dependencias.....
   4.2 Forwarding.....
   4.3 Stalling (load-use).....
   4.4 Flushing (branch).....
5. Dual-Issue preliminar (Dual Fetch / Dual Decode).....
   5.1 Cambios en Fetch y Decode.....
   5.2 Módulos incorporados.....
   5.3 Datapath Dual-Issue preliminar.....
   5.4 Evidencia de Dual Fetch / Dual Decode.....
6. Anexo: programas de prueba.....
   Forward……
   ……
   Stall……
   ……
   Flush……
   ……

1. Objetivo de E1

En esta primera entrega nos propusimos adaptar el procesador RV321 single cycle hacia una arquitectura pipeline de cinco etapas (IF, ID, EX, MEM, WB) donde se le agregó Forwarding, Stalling y Flushing. De esta forma, mantenemos la funcionalidad (pudiendo cargar el mismo archivo de instrucciones) del modelo original mientras establecemos la infraestructura para la futura ejecución superescalar. Adicionalmente, abordamos la fase inicial de la arquitectura Dual-Issue mediante el diseño e integración de los módulos de Dual Fetch y Dual Decode.

2. Pipeline Single-Issue de cinco etapas

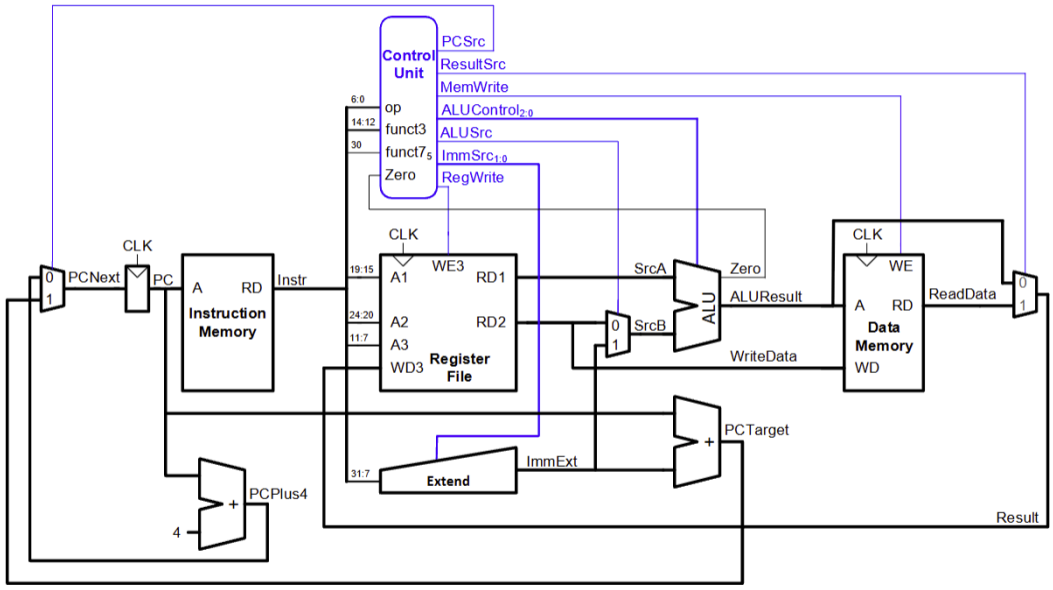
Partimos de la estructura del data path single cycle y la segmentamos en cinco etapas consecutivas separadas por cuatro registros de desacoplamiento ($IF/ID, ID/EX, EX/MEM, MEM/WB$). Este diseño nos permite procesar hasta cinco instrucciones en simultáneo (una por etapa), incrementando la eficiencia hacía aproximadamente una instrucción completada por ciclo de reloj.

2.1 Las cinco etapas

- IF (Fetch): la memoria de instrucciones entrega la instrucción apuntada por el PC; se calcula PC+4.
- ID (Decode): se decodifica la instrucción, se leen los registros fuente y se genera el inmediato (extend).
- EX (Execute): la ALU opera; además se resuelven los saltos (branch/jump).
- MEM (Memory): acceso a la memoria de datos (lw/sw).
- WB (Writeback): se escribe el resultado en el Register File.

2.2 Datapath por etapas

Módulos principales por etapa: IF (PC, imem, sumador +4); ID (regfile, extend); EX (muxes, ALU, sumador de branch, resolución PCSrcE); MEM (dmem); WB (mux de resultado). El control viaja empaquetado dentro de los registros de pipeline y se usa en la etapa correspondiente.



2.3 Unidad de control

Configuramos la unidad de control ($maindec + aludec$) para operar en la etapa ID. A diferencia del esquema singlecycle, no calculamos la señal $PCSrc$ directamente en esta etapa; optamos por resolver los saltos en la etapa EX, motivo por el cual propagamos las señales $Branch$ y $Jump$ de manera independiente a través del pipeline. Asimismo, ajustamos el comportamiento por defecto en la decodificación hacia cero (un NOP) para prevenir estados indeterminados ('x')

3. Hazard Unit (cambios del Single-Issue)

Diseñamos e implementamos la unidad de control de riesgos (hazardunit.v) como un módulo combinacional encargado de gestionar las dependencias de datos y de control. Estructuramos su desarrollo mediante las siguientes estrategias:

3.1 Register File en flanco de bajada

Sincronizamos el Banco de Registros (regfile) para efectuar las escrituras (WB) en el flanco de bajada del reloj ($negedge$) y las lecturas (ID) en el flanco de subida ($posedge$). Esta técnica de *write-before-read* nos permite resolver automáticamente las dependencias a distancia 3 (cuando la etapa WB escribe en el mismo ciclo en que la etapa ID requiere leer dicho registro), evitando añadir adelantamiento adicionales.

3.2 Forwarding (data hazard)

- Se conectan Rs1E/Rs2E (los registros que la instrucción en EX va a leer).
- Dos muxes 3:1 en las entradas de la ALU: 00 = regfile, 10 = ALUResultM (EX/MEM), 01 = ResultW (MEM/WB).
- La Forwarding Unit compara Rs1E/Rs2E contra RdM/RdW; si coinciden (y esa instrucción escribe y no es x0), adelanta el dato. Prioridad al más reciente (MEM sobre WB).

Resuelve las dependencias a distancia 1 y 2 sin perder ciclos.

3.3 Stall (load-use)

Para resolver el conflicto de datos originado por una instrucción lw seguida inmediatamente por una instrucción consumidora, implementamos un mecanismo de detención ($stall$) en la etapa ID. Evaluamos la siguiente condición:

```
lwStall = ResultSrcE[0] & ((Rs1D == RdE) | (Rs2D == RdE))
```

Al detectarse esta condición, congelamos la actualización del contador de programa y del registro IF/ID ($StallF = 1, StallD = 1$), e inyectamos una burbuja de no-operación en el registro ID/EX mediante la señal $FlushE = 1$. Esto introduce la pausa de un ciclo necesaria para que la red de adelantamiento provea el dato leído desde la memoria.

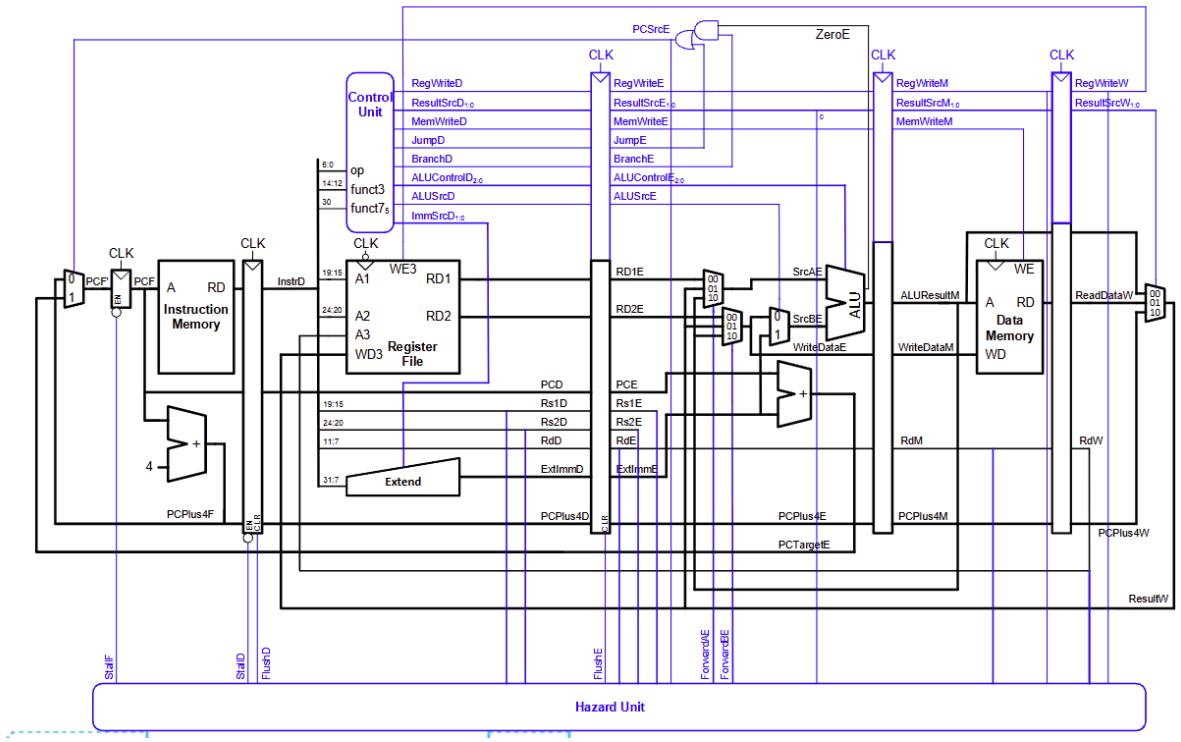
3.4 Flush (control hazard: branch/jump)

Resolvemos las instrucciones de salto en la etapa EX evaluando $PCSrcE = (BranchE \cdot ZeroE) + JumpE$. Cuando confirmamos un salto tomado, procedemos a descartar las dos instrucciones capturadas incorrectamente en las etapas previas activando las señales de limpieza:

```
FlushD = PCSrcE            FlushE = lwStall | PCSrcE
```

Omitimos la señal $lwStall$ en $FlushD$ debido a que durante un congelamiento por carga mantenemos la instrucción consumidora retenida en la etapa de decodificación.

3.5 Diagrama del datapath Single-Issue con Hazard Unit



En el esquema microarquitectónico de nuestro procesador segmentado identificamos claramente las rutas de control e interconexión de riesgos:

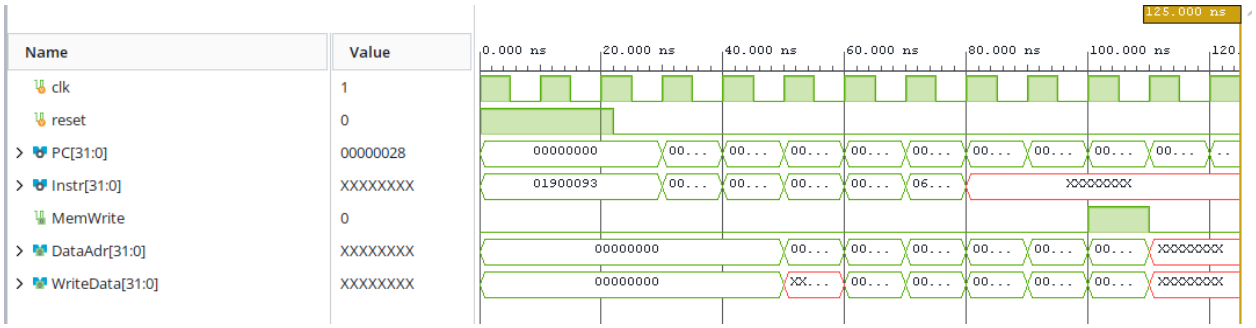
1. **Líneas de Adelantamiento (Forwarding):** Conectan las salidas de los registros de pipeline EX/MEM ($ALUResultM$) y MEM/WB ($ResultW$) hacia los multiplexores de entrada de la ALU en la etapa EX, comandados por las señales ForwardAE y ForwardBE.
2. **Líneas de Congelamiento (Stall):** La salida StallF se conecta a la entrada de habilitación de escritura del registro PC (flopenr), mientras que StallD inhabilita la actualización del registro IF/ID (flopenrc).
3. **Líneas de Limpieza (Flush):** La salida FlushD resetea asíncronamente el contenido de IF/ID, e $FlushE$ limpia el contenido del registro de pipeline ID/EX (floprc), convirtiendo las instrucciones en burbujas NOP.

4. Pruebas: cuatro programas y waveforms

Cada programa aísla un hazard. El testbench verifica el valor esperado en la dirección de memoria 100. Resultados obtenidos en Icarus Verilog:

4.1 Programa sin dependencias

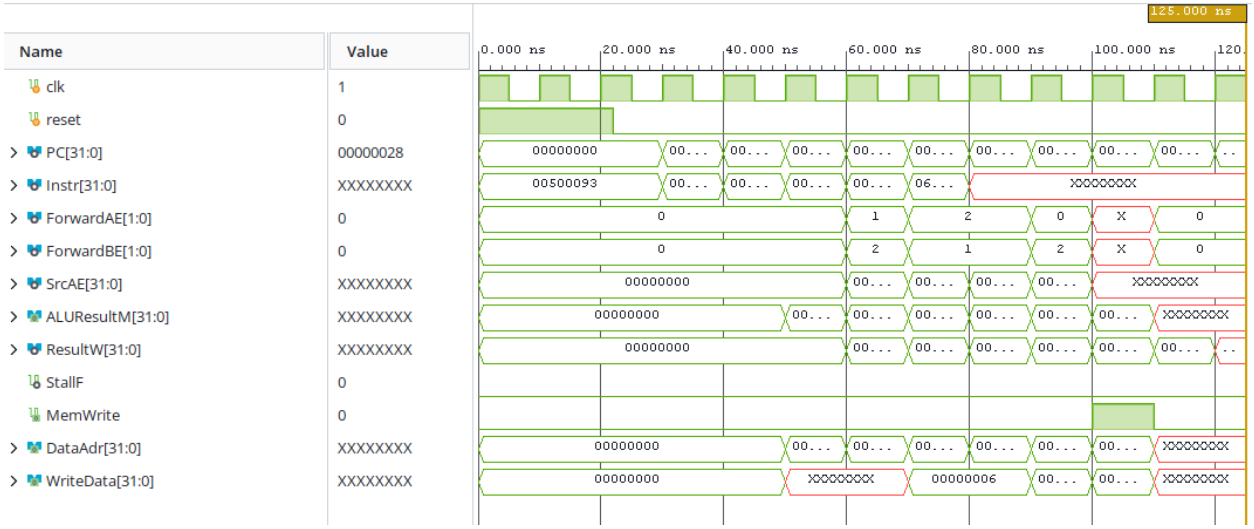
Cinco addi independientes + un sw. Como ninguna instrucción depende de una reciente, no hay hazards y debe dar lo mismo que el single-cycle (Mem[100] = 25).



En la waveform observamos que el PC se incrementa de 4 en 4 en cada ciclo sin interrupciones, y la señal Instr cambia en cada flanco, evidenciando que las instrucciones avanzan por las cinco etapas de forma solapada (una por etapa) sin burbujas. Al no existir dependencias entre instrucciones, no se activa ninguna señal de stall ni de forwarding. Al final, MemWrite se activa una sola vez escribiendo el valor 25 en la dirección 100, idéntico al resultado del single-cycle.

4.2 Forwarding

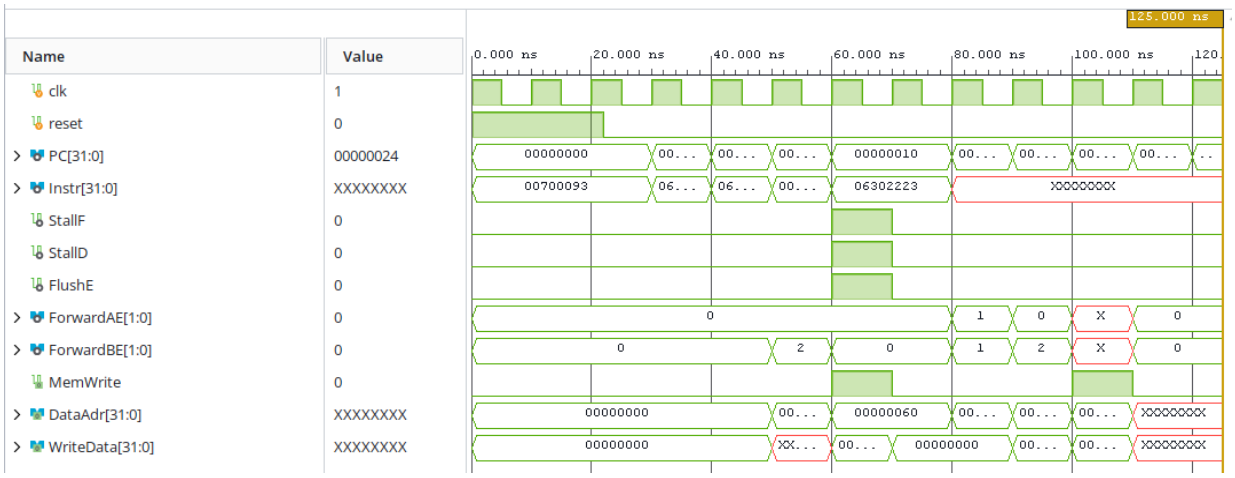
Cadena de sumas dependientes (sin loads). Resultados intermedios esperados: x1=5, x2=6, x3=11, x4=17, x5=28; finalmente Mem[100] = 28. El forwarding adelanta x3 y x4 recién producidos sin frenar el pipeline.



La evidencia del forwarding se observa en las señales ForwardAE y ForwardBE, que toman valores distintos de cero en los ciclos donde se ejecutan instrucciones dependientes: el valor 2 indica adelanto desde la etapa MEM (ALUResultM) y el valor 1 desde WB (ResultW). Por ejemplo, ForwardAE pasa por 1 y 2 al requerir x3 y x4 recién producidos. La señal StallF permanece en 0 durante toda la ejecución, lo que confirma que el forwarding resolvió las dependencias RAW sin introducir burbujas. Finalmente MemWrite escribe 28 en la dirección 100, coincidiendo con el valor esperado.

4.3 Stalling (load-use)

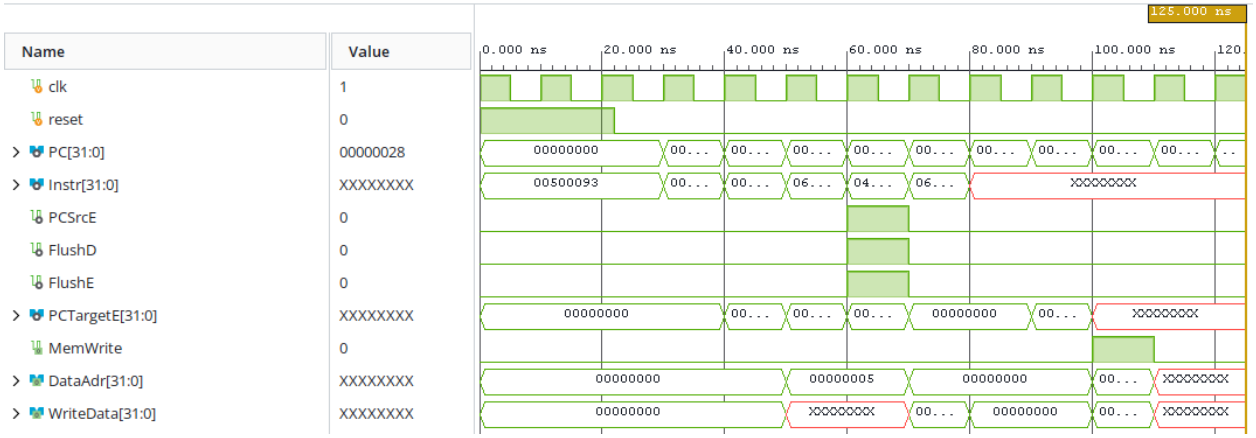
Un lw seguido inmediato de un add que usa el valor cargado. Resultados: x1=7, x2=7 (del lw), x3=14; Mem[100] = 14. Se observa 1 ciclo de burbuja mientras el dato del lw llega.



Observamos que las señales StallF, StallD y FlushE se activan simultáneamente durante un único ciclo : StallF y StallD congelan el PC y el registro IF/ID, mientras FlushE inserta una burbuja en ID/EX. Esa pausa de un ciclo es la evidencia del stall load-use. Tras ella, la red de forwarding (ForwardBE) entrega el dato cargado por el lw. Se aprecian dos escrituras a memoria: la primera corresponde al sw a la dirección 96 y la segunda escribe 14 en la dirección 100.

4.4 Flushing (branch)

Un beq que se toma, con dos instrucciones 'veneno' detrás que no deben ejecutarse. Si el flush funciona, x1 queda intacto y Mem[100] = 5 (si fallara, se escribiría 77).



Al resolverse el branch como tomado en la etapa EX, la señal PCSrcE se activa y, con ella, FlushD y FlushE durante un ciclo, anulando las dos instrucciones capturadas incorrectamente detrás del PC.

Como el flush descartó correctamente esas instrucciones, el registro x1 conserva su valor y MemWrite escribe finalmente 5 en la dirección 100 (si el flush hubiera fallado, se habría escrito 77).

5. Dual-Issue primeros avance (Dual Fetch / Dual Decode)

En esta primera fase abarcamos el diseño e integración del segmento inicial (*front-end*) de la arquitectura Dual-Issue. Nuestro objetivo en esta entrega consiste en validar que la etapa de lectura y decodificación puede duplicarse efectivamente. Dejamos explicitado que la integración de la unidad de emisión (*Issue Unit*), la resolución de conflictos entre vías y la ejecución simétrica simultánea formarán parte del desarrollo de la Entrega E2.

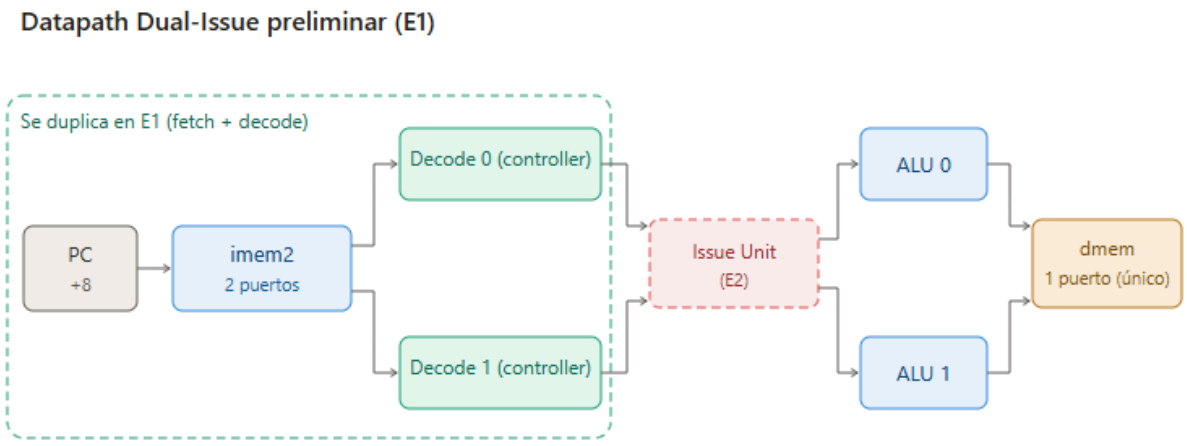
5.1 Cambios en Fetch y Decode

- Dual Fetch: la memoria de instrucciones entrega dos instrucciones por ciclo (PC y PC+4); el PC avanza +8.
- Dual Decode: las dos instrucciones se decodifican en paralelo con dos instancias del controller (módulo reutilizado).

5.2 Módulos incorporados

- imem2.v: Diseñamos una memoria de instrucciones provista de dos puertos de lectura independientes para retornar dos palabras consecutivas en un mismo ciclo.
- dualfetchdecode.v: Desarrollamos el módulo del *front-end* encargado de capturar dos instrucciones simultáneas, calcular el incremento del contador de programa ($PC + 8$) y decodificar ambas vías en paralelo mediante dos unidades de control instanciadas.

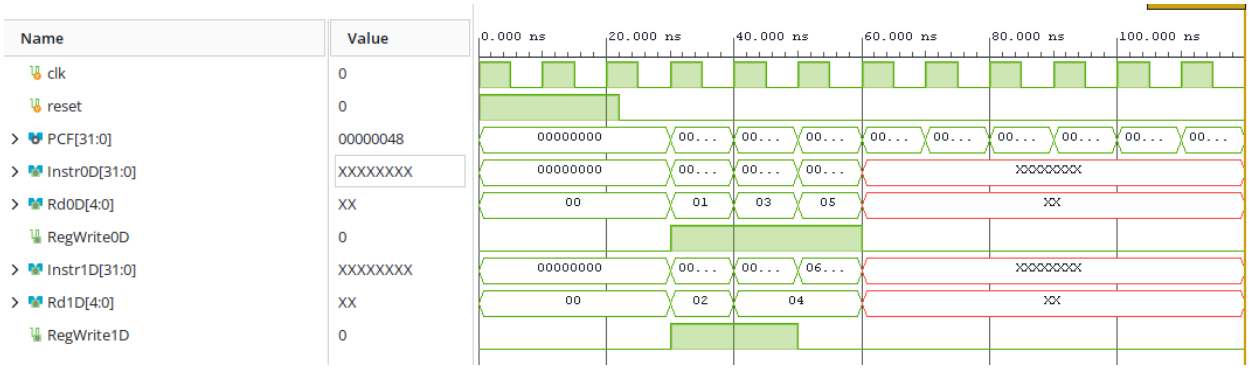
5.3 Datapath Dual-Issue preliminar



Se tiene la base del datapath que recorre el PC en +8 y asigna una instrucción por cada puerto de imagen a los dos módulos de decodificación. Issue Unit por ahora no está implementado.

5.4 Evidencia de Dual Fetch / Dual Decode

El testbench tb_dual_fetch_decode muestra las dos vías decodificando dos instrucciones distintas en el mismo ciclo (por ejemplo, 'addi x1' en la vía 0 y 'addi x2' en la vía 1), con ambos RegWrite en 1.



En la waveform del dual observamos que, en un mismo ciclo, Instr0D e Instr1D contienen dos instrucciones distintas. Los registros lo confirman: Rd0D toma x1, x3, x5 en la vía 0 mientras Rd1D toma x2, x4 en la vía 1, es decir, dos instrucciones se decodifican en paralelo en cada ciclo. Esto evidencia el Dual Fetch + Dual Decode.

6. Anexo: programas de prueba

Forward

Hex	Instrucción
00500093	addi x1, x0, 5
00600113	addi x2, x0, 6
002081B3	add x3, x1, x2
00218233	add x4, x3, x2
003202B3	add x5, x4, x3
06502223	sw x5, 100(x0)

Stall

Hex	Instrucción
00700093	addi x1, x0, 7
06102023	sw x1, 96(x0)
06002103	lw x2, 96(x0)
002101B3	add x3, x2, x2
06302223	sw x3, 100(x0)

Flush

Hex	Instrucción
00500093	addi x1, x0, 5
00500113	addi x2, x0, 5
00208663	beq x1, x2, 12
06300093	addi x1, x0, 99
04D00093	addi x1, x0, 77
06102223	sw x1, 100(x0)